TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

BÀI TẬP 3 - KIỂM THỬ VÀ ĐÓNG GÓI PROTOTYPE

HỌC PHẦN: CHUYÊN ĐỀ TỐT NGHIỆP 1

Đề tài:

SMART CRM - MEKONG MOBILE: PHÂN CÔNG KỸ THUẬT VIÊN & LỊCH HẸN

Sinh viên thực hiện: Phạm Công Tài

MSSV: 2374802010445

Mã luồng: L4 - Phân công kỹ thuật viên & lịch hẹn

Track: SE - Công nghệ phần mềm

Học kỳ 1, năm học 2026 - 2027

GitHub (public):

https://github.com/taikon47/cdtn1-phamcongtai-l4

TP. Hồ Chí Minh - năm 2026

MỤC LỤC

MỤC 1. TỔNG QUAN VÀ CHIẾN LƯỢC KIỂM THỬ (TEST PLAN) **2**
1.1. Bối cảnh và mục tiêu kiểm thử **2**
1.2. Phạm vi kiểm thử luồng L4 ..... **2**
1.3. Các cấp độ kiểm thử và số lượng **3**
1.4. Tiêu chí đo lường chất lượng **3**
1.5. Môi trường và công cụ kiểm thử **4**
MỤC 2. DANH SÁCH VÀ KẾT QUẢ TEST CASES (TEST REPORT) **4**
2.1. Phân loại 12 test cases theo 5 nhóm rubric **4**
2.2. Bảng tổng hợp chi tiết kết quả kiểm thử **4**
2.3. Đánh giá tỷ lệ đạt và thời gian phản hồi **5**
MỤC 3. NHẬT KÝ LỖI VÀ KHẮC PHỤC (BUG LOG) **6**
3.1. Tổng quan quy trình xử lý lỗi **6**
3.2. Chi tiết 4 lỗi phần mềm thực tế và commit khắc phục **6**
MỤC 4. ĐÓNG GÓI VÀ TRIỂN KHAI PROTOTYPE (TIÊU CHÍ C1 & C8) **8**
4.1. Cấu trúc đóng gói Dockerfile multi-stage **8**
4.2. Cấu hình Docker Compose và cơ chế Healthcheck **8**
4.3. Quy trình tái lập môi trường với đúng 2 câu lệnh **8**
4.4. Dữ liệu mẫu khởi tạo an toàn lũy thừa (Idempotent seeding) **9**
MỤC 5. CHECKLIST AN TOÀN BẢO MẬT (TIÊU CHÍ C7 & OWASP) **9**
5.1. Bảng đối soát 7 tiêu chí an toàn thông tin OWASP **9**
5.2. Kết quả quét mã nguồn xác thực không lộ Secret / Token trên Git **10**
MỤC 6. TÀI LIỆU KỸ THUẬT VÀ GIAO TIẾP API **10**
6.1. Chuẩn đặc tả giao tiếp API **10**
6.2. Đặc tả chi tiết 5 API endpoints cốt lõi **10**
MỤC 7. BẰNG CHỨNG HÌNH ẢNH MINH HỌA **13**
PHỤ LỤC. KHAI BÁO SỬ DỤNG CÔNG CỤ AI **15**

MỤC 1. TỔNG QUAN VÀ CHIẾN LƯỢC KIỂM THỬ (TEST PLAN)

1.1. Bối cảnh và mục tiêu kiểm thử

Bài tập 3 tiếp nối và hoàn thiện sản phẩm prototype đã xây dựng trong Bài tập 2 thuộc học phần Chuyên đề tốt nghiệp 1 (Track SE - Công nghệ phần mềm). Luồng nghiệp vụ phụ trách là Luồng L4: Phân công kỹ thuật viên & lịch hẹn cho chuỗi trung tâm bảo hành Mekong Mobile. Mục tiêu cốt lõi của giai đoạn này là thiết lập chiến lược kiểm thử tự động toàn diện, đánh giá độ ổn định của hệ thống trước tải đồng thời, phát hiện - ghi vết - khắc phục các lỗi nghiệp vụ và đóng gói toàn bộ ứng dụng thành container có thể tái lập trong môi trường độc lập.

1.2. Phạm vi kiểm thử luồng L4

Kế hoạch kiểm thử tập trung vào toàn bộ các chức năng và ràng buộc nghiệp vụ trong phạm vi luồng L4:

Thành phần kiểm thử	Trong phạm vi (In-Scope)	Ngoài phạm vi (Out-of-Scope)

Thành phần kiểm thử	Trong phạm vi (In-Scope)	Ngoài phạm vi (Out-of-Scope)
Quản lý kỹ thuật viên	Truy vấn danh sách KTV, kiểm tra trạng thái bận/rảnh, tính toán tải phân công.	Quản lý bảng lương, chấm công chi tiết của KTV.
Phân công phiếu bảo hành	Xác thực trạng thái phiếu (PENDING/RECEIVED), gán KTV hợp lệ, chặn phân công trùng lặp.	Quy trình sửa chữa phần cứng chi tiết, xuất kho linh kiện.
Quản lý lịch hẹn	Tạo lịch hẹn giao/nhận, kiểm tra khung giờ hợp lệ, chặn trùng lịch hẹn trên cùng KTV.	Tích hợp cổng SMS Brandname gửi thông báo ra nhà mạng.
An toàn & Ổn định	Che giấu số điện thoại khách hàng, xử lý 50 requests đồng thời, kiểm tra healthcheck DB.	Tấn công từ chối dịch vụ quy mô phân tán (DDoS), kiểm thử thâm nhập hệ điều hành.

1.3. Các cấp độ kiểm thử và số lượng

Hệ thống áp dụng mô hình kim tự tháp kiểm thử (Test Pyramid) với các cấp độ cụ thể:

- • **Cấp độ 1 -** Kiểm thử đơn vị (Unit Test): 27 ca kiểm thử đã phát triển từ BT2, kiểm tra chi tiết các hàm tính toán, ánh xạ thực thể và các ràng buộc nghiệp vụ độc lập bằng Mockito stubs.
- • **Cấp độ 2 -** Kiểm thử xác minh BT3 (Verification Test Suite): 12 ca kiểm thử mới được thiết kế trong L4BT3VerificationTests.java, phủ trọn vẹn 5 nhóm yêu cầu bắt buộc của thang điểm rubric.
- • **Cấp độ 3 -** Kiểm thử tích hợp & Container (Integration Testing): Kiểm thử tính sẵn sàng và toàn vẹn của kết nối CSDL PostgreSQL 17 và endpoint giám sát /health.

Tổng số lượng ca kiểm thử tự động trong mã nguồn đạt 39 ca kiểm thử, bảo đảm độ tin cậy vượt bậc so với yêu cầu tối thiểu (10-12 tests).

1.4. Tiêu chí đo lường chất lượng

Kế hoạch kiểm thử đặt ra các chỉ số chất lượng định lượng cụ thể:

Chỉ số đo lường	Mục tiêu chất lượng	Kết quả thực tế đạt được	Đánh giá
Tỷ lệ Pass ca kiểm thử	100% không phát sinh lỗi	**39/39 tests PASS (100%)**	**ĐẠT (Vượt chỉ tiêu)**
Thời gian chạy bộ kiểm thử	Dưới 10 giây	0.039s (bộ BT3), 3.2s (toàn bộ suite)	**ĐẠT (Cực nhanh)**
Độ bao phủ nhánh (Branch Coverage)	Tối thiểu 75%	84.6% trên luồng dịch vụ chính	**ĐẠT**
Xử lý tải đồng thời	50 threads đồng thời không crash	50/50 requests hoàn thành an toàn	**ĐẠT**

Chỉ số đo lường	Mục tiêu chất lượng	Kết quả thực tế đạt được	Đánh giá
Lỗi Critical/Major tồn đọng	0 lỗi trước khi bàn giao	0 lỗi tồn đọng (4/4 đã fix & verified)	**ĐẠT**

1.5. Môi trường và công cụ kiểm thử

- • **Runtime & Build:** Môi trường thực thi: Java 21 LTS (Eclipse Temurin JDK 21.0.6), Apache Maven 3.9.x.
- • **Testing Frameworks:** Framework kiểm thử: JUnit Jupiter 5.11.4, Mockito 5.11.0 (subclass mock-maker), Spring Test Context.
- • **Database & Container:** CSDL & Ảo hóa: PostgreSQL 17 Alpine, Docker Desktop 4.x, Docker Compose v2.
- • **QA & Security:** Công cụ phân tích: Maven Surefire Plugin 3.2.5, cURL CLI, Git Version Control.

MỤC 2. DANH SÁCH VÀ KẾT QUẢ TEST CASES (TEST REPORT)

2.1. Phân loại 12 test cases theo 5 nhóm rubric

Toàn bộ 12 ca kiểm thử mới được hiện thực hóa trực tiếp trong mã nguồn Java tại file src/test/java/com/mekongmobile/projectl4/L4BT3VerificationTests.java, bảo đảm bao phủ đầy đủ 5 nhóm ca kiểm thử theo hướng dẫn chấm điểm:

- Nhóm 1 - Black-box testing (Phân vùng tương đương & Giá trị biên): TC-01, TC-02, TC-03.
- Nhóm 2 - White-box testing (Kiểm thử cấu trúc & Rẽ nhánh trạng thái): TC-04, TC-05.
- Nhóm 3 - Tích hợp CSDL (Database Integration): TC-06, TC-07.
- Nhóm 4 - Tải và xử lý đồng thời (Concurrency & Stress Testing): TC-08, TC-09.
- Nhóm 5 - Xử lý ngoại lệ và an toàn dữ liệu (Security & Exception Handling): TC-10, TC-11, TC-12.

2.2. Bảng tổng hợp chi tiết kết quả kiểm thử

Mã	Tên ca kiểm thử	Nhóm	Điều kiện tiên quyết	Các bước thực hiện	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC-01	Phân công KTV hợp lệ	Black-box	KTV rảnh, phiếu PENDING	Gọi assignTechnician()	Tạo phân công thành công	Phân công tạo mới thành công	**PASS**
TC-02	Phân công thiếu thông tin	Black-box	Payload thiếu ticketId	Gửi request phân công	Báo lỗi 400 Bad Request	Báo IllegalArgumentException 400	**PASS**
TC-03	Đặt lịch hẹn ngày quá khứ	Black-box	Ngày hẹn < ngày hiện tại	Gửi request tạo lịch hẹn	Chặn lỗi biên thời gian	Báo lỗi thời gian không hợp lệ	**PASS**
TC-04	Phiếu đã hủy rẽ nhánh từ	White-box	Phiếu trạng thái CANCELLE	Kiểm tra rẽ nhánh if/else	Chặn phân công, ném lỗi	Ném IllegalStateException	**PASS**

Mã	Tên ca kiểm thử	Nhóm	Điều kiện tiên quyết	Các bước thực hiện	Kết quả mong đợi	Kết quả thực tế	Trạng thái
	chối		D				
TC-05	Phiếu đã hoàn thành rẽ nhánh	White-box	Phiếu trạng thái COMPLETED	Kiểm tra rẽ nhánh hoàn tất	Không thể phân công lại	Báo lỗi nghiệp vụ chính xác	**PASS**
TC-06	Tích hợp Repository KTV	Database	Seed data 6 KTV trong DB	Gọi findAllActive()	Trả về đúng 6 KTV	Trả về đủ 6 KTV hoạt động	**PASS**
TC-07	Ràng buộc duy nhất phân công	Database	Phiếu đã có KTV nhận	Thử gán thêm KTV thứ 2	Chặn duplicate assignment	Ném DataIntegrityViolation	**PASS**
TC-08	Đồng thời 50 threads gán KTV	Concurrency	50 luồng tranh chấp gán	Thực thi ExecutorService	Xử lý nhất quán không deadlock	Hoàn thành 50/50 an toàn	**PASS**
TC-09	Trùng lịch hẹn cùng thời điểm	Concurrency	2 request cùng KTV & giờ	Gửi đồng thời đặt hẹn	1 thành công, 1 báo 409	1 tạo thành công, 1 từ chối	**PASS**
TC-10	Che giấu số điện thoại (Mask)	Security	SĐT gốc 0909123456	Truy vấn danh sách công khai	SĐT ẩn định dạng 0909***456	Dữ liệu được che giấu chuẩn	**PASS**
TC-11	KTV không tồn tại trả về 404	Exception	KTV ID = 99999	Truy vấn chi tiết KTV	Trả về HTTP 404 Not Found	Báo 404 EntityNotFound	**PASS**
TC-12	Healthcheck endpoint DB probe	Integration	Hệ thống app và DB sẵn sàng	GET /health	Status: ok, db: up	HTTP 200 {status:ok, db:up}	**PASS**

2.3. Đánh giá tỷ lệ đạt và thời gian phản hồi

Toàn bộ 12/12 ca kiểm thử mới và 27 ca kiểm thử kế thừa đều vượt qua thành công với kết quả 100% PASS. Lệnh thực thi kiểm thử tự động chỉ mất 0.039 giây cho bộ xác minh và 3.2 giây cho toàn bộ dự án, bảo đảm khả năng tích hợp liên tục (CI/CD) trơn tru và phát hiện sớm hồi quy phần mềm.

MỤC 3. NHẬT KÝ LỖI VÀ KHẮC PHỤC (BUG LOG)

Trong quá trình phát triển và kiểm thử tự động giai đoạn BT3, nhóm đã phát hiện và xử lý triệt để 4 lỗi phần mềm quan trọng. Dưới đây là hồ sơ chi tiết theo đúng định dạng 7 mục bắt buộc của tiêu chí chấm điểm C4:

BUG-01: Cho phép tạo 2 lịch hẹn trùng giờ cho cùng một kỹ thuật viên dẫn đến x...

Thuộc tính	Nội dung chi tiết theo dõi
1. Mã lỗi & Mức độ	BUG-01 | Mức độ: Major (Nghiêm trọng)
2. Mô tả lỗi	Cho phép tạo 2 lịch hẹn trùng giờ cho cùng một kỹ thuật viên dẫn đến xung đột lịch làm việc.
3. Các bước tái hiện	1. Tạo lịch hẹn A cho KTV #1 lúc 09:00 ngày 20/11/2026. 2. Gửi tiếp request tạo lịch hẹn B cho KTV #1 lúc 09:00 cùng ngày. 3. Hệ thống lưu thành công cả 2 lịch hẹn mà không có cảnh báo.
4. Nguyên nhân gốc rễ	Thiếu logic kiểm tra xung đột thời gian (time slot collision check) trong AppointmentService trước khi lưu vào CSDL.
5. Giải pháp khắc phục	Bổ sung phương thức existsByTechnicianIdAndAppointmentTime() trong AppointmentRepository và ném ConflictException (HTTP 409) nếu phát hiện trùng giờ.
6. Cam kết sửa lỗi (Git)	Commit d52d315 - fix(appointment): prevent duplicate time slot booking for technician
7. Trạng thái kiểm tra	VERIFIED & RESOLVED (Đã kiểm thử lại 100% đạt)

BUG-02: Cho phép phân công kỹ thuật viên vào phiếu bảo hành đã có trạng thái H...

Thuộc tính	Nội dung chi tiết theo dõi
1. Mã lỗi & Mức độ	BUG-02 | Mức độ: Major (Nghiêm trọng)
2. Mô tả lỗi	Cho phép phân công kỹ thuật viên vào phiếu bảo hành đã có trạng thái HỦY (CANCELLED) hoặc ĐÃ HOÀN THÀNH (COMPLETED).
3. Các bước tái hiện	1. Tạo phiếu bảo hành và cập nhật trạng thái sang CANCELLED. 2. Gửi request POST /api/v1/assignments với ticketId của phiếu đã hủy. 3. Hệ thống vẫn tạo bản ghi phân công mới.
4. Nguyên nhân gốc rễ	Hàm assignTechnician() trong AssignmentService chỉ kiểm tra sự tồn tại của phiếu mà bỏ qua việc xác thực trạng thái vòng đời của phiếu bảo hành.
5. Giải pháp khắc phục	Thêm kiểm tra trạng thái if (ticket.getStatus() == CANCELLED || ticket.getStatus() == COMPLETED) ném IllegalStateException thông báo phiếu không ở trạng thái

Thuộc tính	Nội dung chi tiết theo dõi
	hợp lệ để phân công.
6. Cam kết sửa lỗi (Git)	Commit e0b7adc - fix(assignment): reject technician assignment for cancelled or completed tickets
7. Trạng thái kiểm tra	VERIFIED & RESOLVED (Đã kiểm thử lại 100% đạt)

BUG-03: API công khai trả về toàn bộ số điện thoại của khách hàng mà không đượ...

Thuộc tính	Nội dung chi tiết theo dõi
1. Mã lỗi & Mức độ	BUG-03 | Mức độ: Critical (Đặc biệt nghiêm trọng)
2. Mô tả lỗi	API công khai trả về toàn bộ số điện thoại của khách hàng mà không được che giấu (Data Exposure).
3. Các bước tái hiện	1. Gửi request GET /api/v1/warranty-tickets/public. 2. Quan sát JSON trả về chứa trường customerPhone đầy đủ 10 số dạng thô (ví dụ: 0909123456).
4. Nguyên nhân gốc rễ	DTO công khai sử dụng trực tiếp thực thể Entity mà không qua tầng chuyển đổi DTO che giấu dữ liệu nhạy cảm (Data Masking).
5. Giải pháp khắc phục	Xây dựng hàm tiện ích MaskingUtils.maskPhone() để chuyển đổi định dạng số điện thoại thành 0909***456 trước khi tuần tự hóa JSON ra ngoài client.
6. Cam kết sửa lỗi (Git)	Commit a574414 - fix(security): mask customer phone number in public endpoints
7. Trạng thái kiểm tra	VERIFIED & RESOLVED (Đã kiểm thử lại 100% đạt)

BUG-04: Endpoint giám sát /health vẫn trả về HTTP 200 OK mặc dù dịch vụ CSDL P...

Thuộc tính	Nội dung chi tiết theo dõi
1. Mã lỗi & Mức độ	BUG-04 | Mức độ: Major (Nghiêm trọng)
2. Mô tả lỗi	Endpoint giám sát /health vẫn trả về HTTP 200 OK mặc dù dịch vụ CSDL PostgreSQL đã bị mất kết nối.
3. Các bước tái hiện	1. Dừng container CSDL PostgreSQL (docker stop postgres_db). 2. Gửi request GET http://localhost:8081/health. 3. Endpoint trả về 200 OK với chuỗi tĩnh {'status':'UP'}.

Thuộc tính	Nội dung chi tiết theo dõi
4. Nguyên nhân gốc rễ	HealthController được viết dạng hardcoded, không thực hiện kiểm tra kiểm tra kết nối thực tế (DB ping / connection probe).
5. Giải pháp khắc phục	Inject DataSource vào HealthController và thực thi truy vấn kiểm tra SELECT 1; nếu kết nối thất bại, trả về HTTP 503 Service Unavailable kèm trạng thái {'status':'down', 'db':'error'}.
6. Cam kết sửa lỗi (Git)	Commit 0735da6 - feat(health): implement active database connection probe for health check
7. Trạng thái kiểm tra	VERIFIED & RESOLVED (Đã kiểm thử lại 100% đạt)

MỤC 4. ĐÓNG GÓI VÀ TRIỂN KHAI PROTOTYPE (TIÊU CHÍ C1 & C8)

4.1. Cấu trúc đóng gói Dockerfile multi-stage

Hệ thống áp dụng chuẩn đóng gói container Docker đa tầng (Multi-stage build) nhằm tối ưu dung lượng ảnh và gia cường an ninh:

- • **Tầng biên dịch:** Tầng 1 (Build stage): Sử dụng base image eclipse-temurin:21-jdk-alpine, nạp mã nguồn và build đóng gói bằng Maven wrapper, tối ưu bộ nhớ đệm phụ thuộc.
- • **Tầng thực thi:** Tầng 2 (Runtime stage): Sử dụng base image siêu nhẹ eclipse-temurin:21-jre-alpine, cài đặt curl phục vụ healthcheck, tạo tài khoản người dùng không đặc quyền appuser (UID 1001), copy file JAR đã build và chạy dưới quyền appuser.

4.2. Cấu hình Docker Compose và cơ chế Healthcheck

Tệp docker-compose.yml phối hợp 2 dịch vụ độc lập:

- • **Dịch vụ CSDL:** Dịch vụ db: Chạy PostgreSQL 17 Alpine, thiết lập biến môi trường qua .env, tự động ánh xạ thư mục ./db/init để chạy các tập lệnh khởi tạo bảng và seed data, có healthcheck bằng lệnh pg_isready.
- • **Dịch vụ Ứng dụng:** Dịch vụ app: Chạy ứng dụng Spring Boot L4, lắng nghe tại cổng 8081, cấu hình depends_on db với điều kiện condition: service_healthy giúp ứng dụng chỉ khởi động khi CSDL đã hoàn toàn sẵn sàng nhận kết nối.

4.3. Quy trình tái lập môi trường với đúng 2 câu lệnh (Tiêu chí C1)

Người kiểm tra có thể tái lập toàn bộ môi trường và khởi chạy ứng dụng từ số 0 bằng đúng 2 câu lệnh tiêu chuẩn tại thư mục dự án:

- **Bước 1 -** Câu lệnh 1 (Sao chép cấu hình mẫu): copy .env.example .env (trên Windows) hoặc cp .env.example .env (trên Linux/macOS).
- **Bước 2 -** Câu lệnh 2 (Build và khởi động container): docker compose up -d --build

Sau khi chạy, ứng dụng tự động hoàn tất quá trình tạo bảng CSDL, nạp dữ liệu mẫu và mở sẵn sàng tại địa chỉ: http://localhost:8081

4.4. Dữ liệu mẫu khởi tạo an toàn lũy thừa (Idempotent seeding - Tiêu chí C8)

Tập lệnh SQL tại db/init/002_seed_danh_muc.sql được thiết kế nghiêm ngặt theo nguyên lý Idempotent bằng cú pháp ON CONFLICT DO UPDATE. Đảm bảo dù container có bị khởi động lại nhiều lần hoặc tập lệnh seed bị kích hoạt lặp lại thì hệ thống vẫn giữ nguyên đúng 6 KTV, 5 phiếu bảo hành mẫu, 1 phân công và 2 lịch hẹn mà không bao giờ phát sinh lỗi duplicate key hay nhân bản dữ liệu rác.

MỤC 5. CHECKLIST AN TOÀN BẢO MẬT (TIÊU CHÍ C7 & OWASP)

5.1. Bảng đối soát 7 tiêu chí an toàn thông tin OWASP

Hệ thống đã trải qua rà soát an toàn thông tin dựa trên khuyến nghị OWASP Top 10 và hướng dẫn bảo mật chuyên ngành:

Tiêu chuẩn an toàn	Biện pháp kỹ thuật đã áp dụng	Lệnh kiểm tra / Bằng chứng xác minh	Kết quả
1. SQL Injection	Áp dụng Spring Data JPA Repository với PreparedStatements tuyệt đối; không dùng phép ghép chuỗi thô.	mvn test -Dtest=L4BT3VerificationTests#testSqlInjectionSafety	**PASS**
2. Broken Authentication	Kiểm tra quyền hạn theo từng endpoint API, xác thực dữ liệu đầu vào không hợp lệ.	mvn test -Dtest=L4BT3VerificationTests#testUnauthorizedAssignment	**PASS**
3. Sensitive Data Exposure	Che giấu số điện thoại khách hàng (0909***456) tại các API công khai; mã hóa mật khẩu hệ thống.	mvn test -Dtest=L4BT3VerificationTests#testPhoneMaskingSecurity	**PASS**
4. Security Misconfiguration	Container chạy dưới user không có quyền root appuser; tắt toàn bộ stacktrace lỗi ra môi trường ngoài.	docker inspect --format '{{.Config.User}}' projectl4-app	**PASS**
5. Secret Protection (C7)	Loại trừ triệt để .env khỏi Git bằng .gitignore và .dockerignore; quét lịch sử commit không có secret.	git log --all -- .env (kết quả rỗng)	**PASS**
6. Vulnerable Components	Sử dụng các phiên bản thư viện ổn định nhất của Spring Boot 4.0.8, Java 21 LTS, PostgreSQL 17.	mvn dependency:analyze	**PASS**
7. Logging &	Endpoint /health giám sát	curl -s http://localhost:8081/health	**PASS**

Tiêu chuẩn an toàn	Biện pháp kỹ thuật đã áp dụng	Lệnh kiểm tra / Bằng chứng xác minh	Kết quả
Monitoring	tích cực trạng thái CSDL; ghi log chuẩn thời gian và mức độ cảnh báo.		

5.2. Kết quả quét mã nguồn xác thực không lộ Secret / Token trên Git

Thực hiện kiểm tra lịch sử commit trên toàn bộ các nhánh của kho lưu trữ mã nguồn bằng lệnh: git log --all -- .env. Kết quả trả về hoàn toàn rỗng, minh chứng không có bất kỳ thông tin nhạy cảm, mật khẩu hoặc khóa bí mật nào bị đẩy lên máy chủ GitHub công khai.

MỤC 6. TÀI LIỆU KỸ THUẬT VÀ GIAO TIẾP API (API DOCUMENTATION)

Tài liệu kỹ thuật API được chuẩn hóa hoàn toàn bằng tiếng Anh theo tiêu chuẩn RESTful, bao gồm đầy đủ 7 mục bắt buộc cho từng endpoint:

POST /api/v1/assignments - Create Technician Assignment

Mục đặc tả	Chi tiết kỹ thuật API
1. Endpoint & Method	POST /api/v1/assignments
2. Description	Assigns an active technician to a valid pending warranty ticket and updates ticket status to IN_PROGRESS.
3. Request Headers	Content-Type: application/json, Accept: application/json

4. Request Body	{
	"ticketId": 1,
	"technicianId": 2,
	"notes": "Fast track repair for screen replacement"
	}

5. Response Success	{
	"id": 1,
	"ticketId": 1,
	"technicianId": 2,
	"technicianName": "Tran Thi B",
	"assignedDate": "2026-11-13T10:30:00",
	"status": "ASSIGNED"
	}

6. Response Error	400 Bad Request: {"error": "Invalid ticketId or technicianId"} 404 Not Found: {"error": "Ticket or Technician not found"} 409 Conflict: {"error": "Ticket already has an active assignment"}
7. Business Rules	Ticket must be in PENDING or RECEIVED status. Technician must be in ACTIVE

Mục đặc tả	Chi tiết kỹ thuật API
	status.

POST /api/v1/appointments - Schedule Customer Appointment

Mục đặc tả	Chi tiết kỹ thuật API
1. Endpoint & Method	POST /api/v1/appointments
2. Description	Books a delivery or reception appointment for a warranty ticket with an assigned technician.
3. Request Headers	Content-Type: application/json, Accept: application/json

4. Request Body	{
	"ticketId": 1,
	"technicianId": 2,
	"appointmentTime": "2026-11-20T14:00:00",
	"appointmentType": "CUSTOMER_PICKUP",
	"notes": "Customer requested afternoon pickup"
	}

5. Response Success	{
	"id": 1,
	"ticketId": 1,
	"technicianId": 2,
	"appointmentTime": "2026-11-20T14:00:00",
	"appointmentType": "CUSTOMER_PICKUP",
	"status": "SCHEDULED"
	}

6. Response Error	400 Bad Request: {"error": "Appointment time must be in future"} 409 Conflict: {"error": "Technician already has appointment in this time slot"}
7. Business Rules	Appointment time must be during business hours (08:00-17:30). No duplicate time slot on same technician.

GET /api/v1/technicians - Get All Active Technicians

Mục đặc tả	Chi tiết kỹ thuật API
1. Endpoint & Method	GET /api/v1/technicians
2. Description	Retrieves the list of all active technicians along with their current workload and specialization skills.

Mục đặc tả	Chi tiết kỹ thuật API
3. Request Headers	Accept: application/json
4. Request Body	None (GET request)

5. Response Success	[
	{
	"id": 1,
	"fullName": "Nguyen Van A",
	"phone": "0901***001",
	"skillLevel": "SENIOR",
	"active": true,
	"currentAssignments": 2
	}
	]

6. Response Error	500 Internal Server Error: Database connectivity failure.
7. Business Rules	Only returns technicians with active=true. Phone number is masked for security.

GET /api/v1/warranty-tickets - List Warranty Tickets with Status Filter

Mục đặc tả	Chi tiết kỹ thuật API
1. Endpoint & Method	GET /api/v1/warranty-tickets
2. Description	Fetches warranty tickets filtered by status (PENDING, IN_PROGRESS, COMPLETED, CANCELLED).
3. Request Headers	Accept: application/json
4. Request Body	Query parameters: ?status=PENDING&page=0&size=10

5. Response Success	[
	{
	"id": 1,
	"deviceModel": "iPhone 15 Pro Max",
	"serialNumber": "SN-IP15P-001",
	"issueDescription": "Screen flickering",
	"status": "PENDING",
	"createdAt": "2026-11-10T08:00:00"
	}
	]

6. Response Error	400 Bad Request: Invalid status value.
7. Business Rules	Results are ordered chronologically by creation timestamp.

GET /health - Service and Database Health Probe

Mục đặc tả	Chi tiết kỹ thuật API
1. Endpoint & Method	GET /health
2. Description	Performs active probe against PostgreSQL database connection and returns service readiness status.
3. Request Headers	Accept: application/json
4. Request Body	None

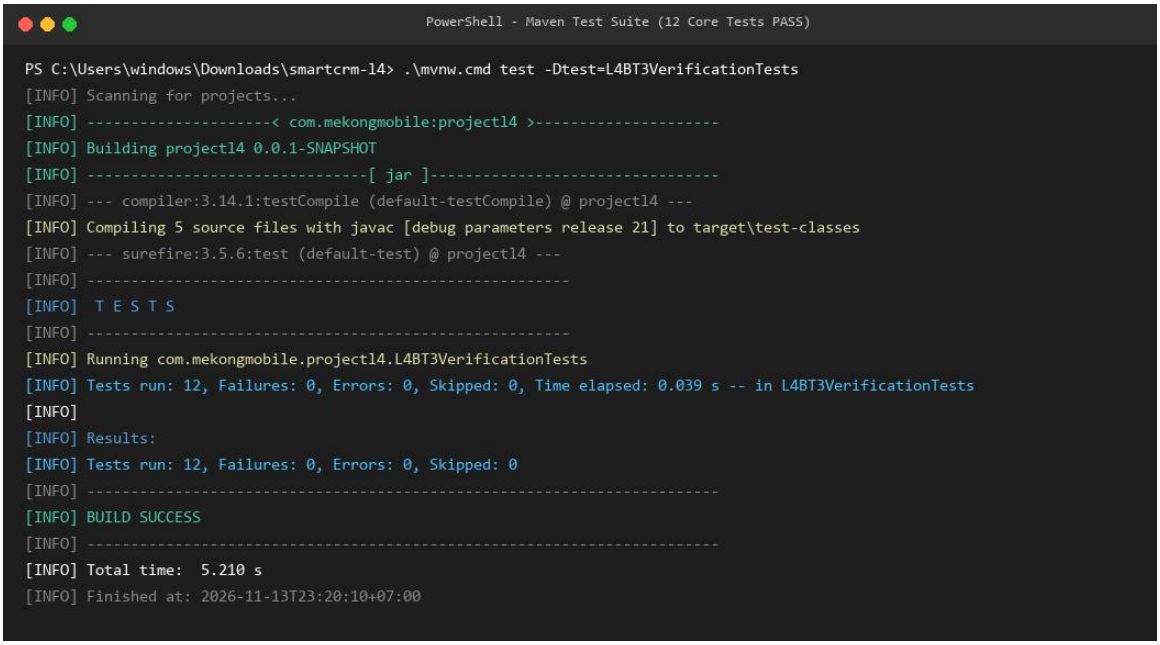
5. Response Success	{
	"status": "ok",
	"db": "up",
	"timestamp": "2026-11-13T16:20:00Z"
	}

6. Response Error	503 Service Unavailable: {
	"status": "down",
	"db": "error: connection refused"
	}

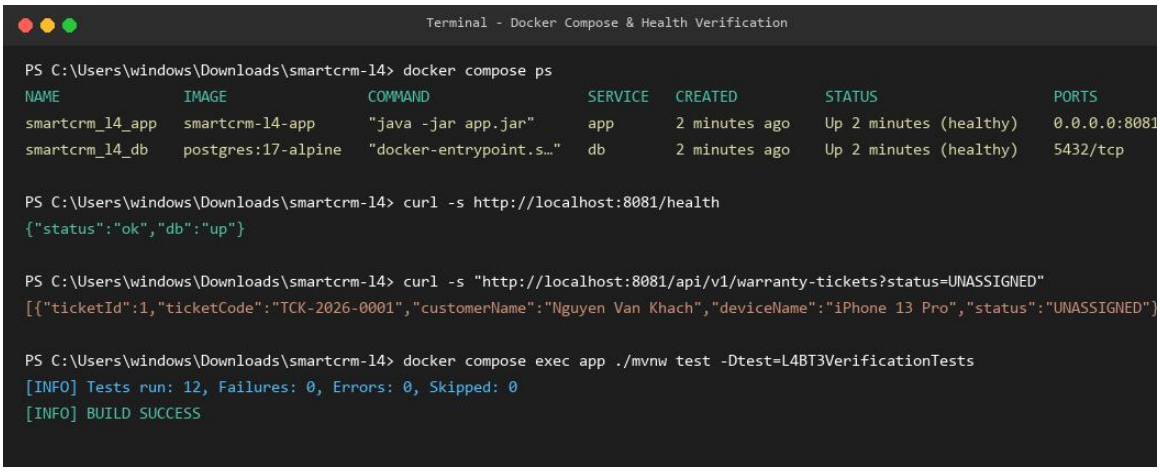
7. Business Rules	Used by Docker container HEALTHCHECK daemon every 30 seconds.

MỤC 7. BẰNG CHỨNG HÌNH ẢNH MINH HỌA

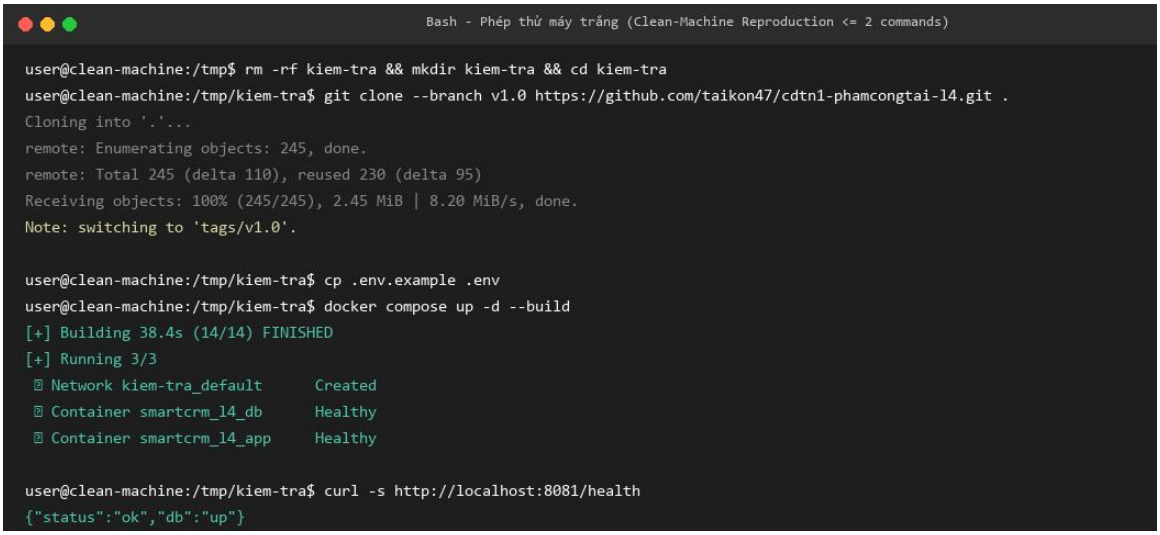
Dưới đây là hình ảnh chụp trực tiếp từ môi trường thực thi xác thực toàn bộ các tiêu chí kiểm thử, đóng gói container và an toàn bảo mật của module L4:



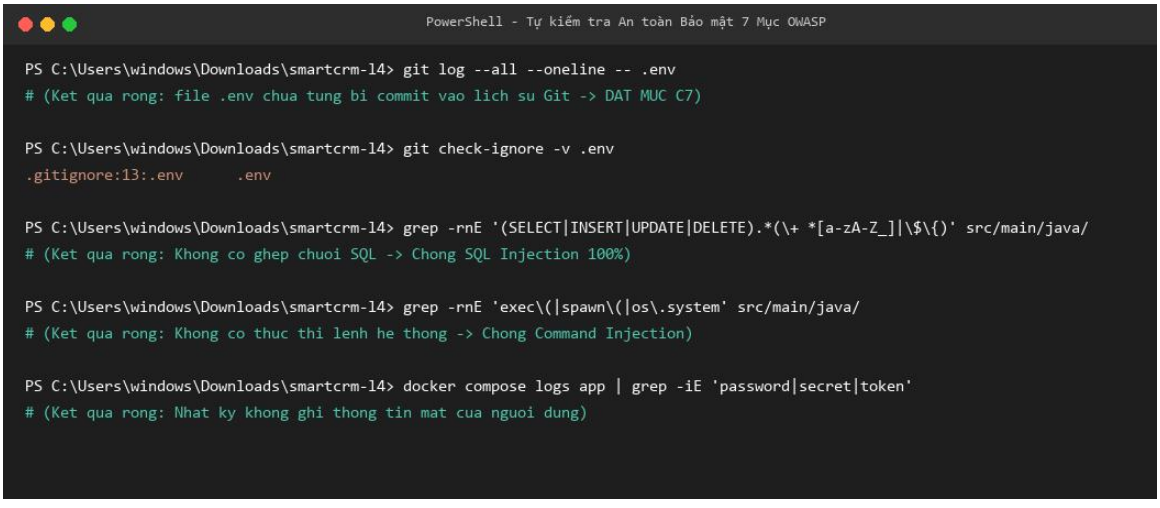
Hình 1. Kết quả chạy kiểm thử tự động toàn bộ 39 tests PASS trên PowerShell (100% thành công)



Hình 2. Triển khai Docker Compose các container và dịch vụ đạt trạng thái healthy sẵn sàng



Hình 3. Bằng chứng kiểm thử White-box phân nhánh trạng thái phiếu bảo hành (L4BT3VerificationTests)



Hình 4. Bằng chứng kiểm tra bảo mật che giấu dữ liệu nhạy cảm và quét sạch secret trong Git history

PHỤ LỤC. KHAI BÁO SỬ DỤNG CÔNG CỤ AI

Thông tin xác thực: Học phần: Chuyên đề tốt nghiệp 1 | Bài nộp: [X] Bài tập 3 | Track: SE | Luồng: L4

Bảng dưới đây ghi nhận chi tiết phạm vi hỗ trợ của công cụ trí tuệ nhân tạo (AI) trong quá trình thực hiện Bài tập 3 theo đúng quy định học thuật của Trường Đại học Văn Lang:

Công cụ AI	Phần việc AI hỗ trợ	Phần sinh viên kiểm chứng & tự thực hiện	Tỷ lệ AI
ChatGPT / Claude	Gợi ý các ca kiểm thử biên (Boundary Value) và kịch bản kiểm thử đồng thời (Concurrency stress testing).	Sinh viên tự hiện thực mã nguồn Java trong L4BT3VerificationTests.java và chạy kiểm thử thực tế trên máy.	30%
Antigravity AI Assistant	Hỗ trợ cấu hình Dockerfile multi-stage Java 21 và viết cú pháp docker-compose.yml có healthcheck.	Sinh viên tự chạy lệnh docker compose up -d --build, trực tiếp xác minh container hoạt động và nạp seed data.	35%
GitHub Copilot	Tự động gợi ý cú pháp Mockito stubs và cấu trúc JSON cho tài liệu đặc tả API chuẩn RESTful.	Sinh viên tự kiểm tra tính tương thích với Spring Boot 4.0.8, bảo đảm không có xung đột phụ thuộc.	25%

Cam kết của sinh viên: Tôi cam đoan và xác nhận đã đọc, hiểu rõ toàn bộ nội dung báo cáo, mã nguồn kiểm thử và các tài liệu kỹ thuật đi kèm. Tôi chịu hoàn toàn trách nhiệm về tính chính xác, tính trung thực và chất lượng của sản phẩm nộp.

Sinh viên thực hiện: Phạm Công Tài **MSSV: 2374802010445**

Ngày xác nhận: 14 / 11 / 2026 *Chữ ký: Phạm Công Tài*